Índices compuestos y consultas SELECT

Índices compuestos, filtros en una tienda web y estructura de una consulta completa

Semana 10 - Sesión 19

Rel Guzman



Logro de la sesión

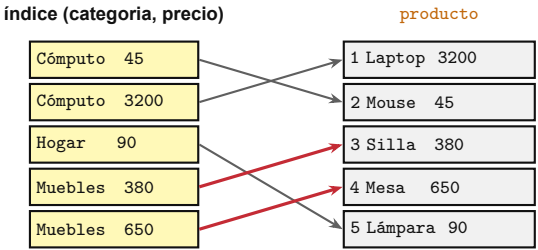
Al finalizar, podrás decidir cuándo conviene un **índice de varias columnas**, elegir el orden de sus columnas y leer la estructura de una **consulta** `SELECT` **completa**.

Revisión de la sesión anterior: en resumen

	En una frase	Ejemplo
Índice	Una lista ordenada que dice dónde está cada fila.	`CREATE INDEX idx_cliente_nombre ON cliente (nombre);`
`IS NULL`	Pregunta si falta el dato. Nunca = `NULL`.	`WHERE telefono IS NULL`
`COALESCE` / `NULLIF`	Un repuesto para el `NULL`; convertir un valor en `NULL`.	`COALESCE(telefono, 'sin teléfono')`
Expresión	Un cálculo fila por fila, con su `AS`.	`sueldo * 12 AS sueldo_anual`
`CASE WHEN`	Decide un valor según una condición.	`CASE WHEN sueldo >= 3000 THEN 'alto' ELSE 'bajo' END`
Agregación	Muchas filas, un solo valor.	`COUNT(*), SUM, AVG, MIN, MAX`
`GROUP BY` / `HAVING`	Un resultado por grupo; filtrar los grupos.	`GROUP BY area HAVING COUNT(*) >= 2`

Hoy repasamos dos ideas más a fondo: los **índices de varias columnas** y la **consulta `SELECT` completa**.

Revisión: un índice de varias columnas



- Se ordena primero por `categoria`.
- Si dos tienen la **misma** categoría, se ordenan por `precio`.
- «Muebles de menos de 700» están **juntos** en la lista.

```
CREATE INDEX idx_producto_categoria_precio
   ON producto (categoria, precio);
```

En palabras simples

Es la **guía telefónica**: ordenada por apellido y, dentro del mismo apellido, por nombre.

Revisión: el orden de las columnas importa

Con el índice `(categoria, precio)`, ¿qué consultas salen rápido?

La consulta tiene…	¿Lo usa?	Por qué
`WHERE categoria = 'Muebles'`	Sí	usa la **primera** columna
`WHERE categoria = 'Muebles' AND precio < 700`	Sí	usa las **dos** columnas
`WHERE categoria = 'Muebles' ORDER BY precio`	Sí	ya vienen **ordenados** por precio
`WHERE precio < 700`	No	se **saltó** la primera columna

La regla

El índice se usa **de izquierda a derecha**. Pon primero la columna que se compara con =, y después la del rango o la del `ORDER BY`.

En palabras simples

En la guía buscas rápido a los «Pérez» o a «Pérez, Ana». Pero no a todas las «Ana»: están repartidas por toda la guía.

Revisión: una tienda web con filtros



Cada filtro que elige el cliente se vuelve **una parte** de la consulta:

```sql
SELECT nombre, precio
FROM producto
WHERE categoria = 'Hogar'
   AND precio BETWEEN 2000 AND 2500
ORDER BY precio
LIMIT 10;
```

Esta consulta se repite **cada vez** que alguien filtra. El índice que la acelera:

```sql
CREATE INDEX idx_producto_categoria_precio
   ON producto (categoria, precio);
```

Primero `categoria`, que va con =; después `precio`, que es un rango y además ordena.

Demostración: la tienda con un millón de productos

La carpeta `tienda/` trae esa página. Con los botones **creas y borras** el índice, y comparas.

Crear índice

```
CREATE INDEX idx_producto_categoria_precio
   ON producto (categoria, precio);
```

Borrar índice

```
DROP INDEX idx_producto_categoria_precio;
```

	Sin índice	Con índice
El filtro tarda	_____ ms	_____ ms
«Cómo buscó» dice	`SCAN producto`	`SEARCH … USING INDEX`

`SCAN` es leer la tabla **completa**, fila por fila. `SEARCH` es ir **directo** con el índice.

Para probarlo, en la carpeta `tienda/`: `npm install`, `npm run crear` y `npm run dev`. Luego abre `http://localhost:3003`.

Práctica 1 ¿Índice? ¿Sobre qué columnas?

La tabla `pedido(id, cliente_id, fecha, estado, total)` de una tienda en línea tiene **dos millones** de filas.

Situación	¿Índice? ¿Sobre qué?
1. La página «Mis pedidos»: `WHERE cliente_id = ? ORDER BY fecha`	
2. El almacén ve los pendientes: `WHERE estado = 'pendiente' AND fecha >= ?`	
3. Abrir un pedido por su número: `WHERE id = ?`	
4. Un reporte que se corre **una vez al año**: `WHERE total > 1000`	
5. La tabla `metodo_pago` tiene solo 3 filas: efectivo, tarjeta, Yape.	
6. Ya existe `(cliente_id, fecha)`. ¿Sirve para `WHERE fecha = ?`	

Para escribir: el `CREATE INDEX` de las situaciones 1 y 2. **Pista:** el número de pedido es la clave primaria.

Revisión: todo junto, lo que puede llevar un `SELECT`

```sql
SELECT c.nombre, COUNT(*) AS pedidos
FROM cliente c
JOIN pedido p ON c.id = p.cliente_id
WHERE p.fecha >= '2026-01-01'
GROUP BY c.nombre
HAVING COUNT(*) >= 2
ORDER BY pedidos DESC
LIMIT 5;
```

- qué columnas o cálculos se **muestran**
- la tabla **principal**
- qué otra tabla se **une**, y por dónde
- qué **filas** entran
- cómo se **agrupan**
- qué **grupos** se quedan
- en qué **orden** salen
- cuántas filas, **como máximo**

- `SELECT` indica qué mostrar. `FROM` se usa al consultar tablas. Lo demás se agrega si hace falta.
- Si se usan, se escriben **siempre en este orden**.
- `SELECT DISTINCT` quita las filas **repetidas** del resultado.

Se lee: «los 5 clientes con más pedidos este año, entre los que tienen 2 o más».

Cierre: lo de hoy en una tabla

	En una frase
Índice de varias columnas	Ordena por la primera columna y, dentro de ella, por la siguiente.
Orden de las columnas	Importa el prefijo izquierdo del índice.
Tienda con filtros	Comparamos el plan y el tiempo de consulta con y sin índice.
`SELECT` **completo**	Cada cláusula tiene una función y un lugar al escribir la consulta.
`WHERE` / `HAVING`	Filtran filas y grupos, respectivamente.
`ORDER BY` / `LIMIT`	Ordenan el resultado y limitan la cantidad de filas.

El índice se elige según las **consultas que hace la aplicación**.

Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.

Microsoft. (s. f.). *Transact-SQL reference*. `https://learn.microsoft.com/sql/t-sql/language-reference`

Microsoft. (s. f.). *SQL Server Management Studio (SSMS)*. `https://learn.microsoft.com/sql/ssms`

Oracle. (s. f.). *MySQL 8.0 reference manual: multiple-column indexes*. `https://dev.mysql.com/doc/refman/8.0/en/multiple-column-indexes.html`

SQLite. (s. f.). *Query planning*. `https://www.sqlite.org/queryplanner.html`



UTP Universidad Tecnológica del Perú